

Générateur de Pouvoir — Description du projet (V2)

Table of Contents

1. Vue d'ensemble	2
2. Déterminisme — Seed unique	2
3. Concepts du domaine	3
3.1. Traits	3
3.2. Pouvoirs	3
3.3. Personnes	4
3.4. Espèces et genres	4
3.5. ADN	5
4. Hérité — Modèle de résilience	5
4.1. Principe	5
4.2. Combinaison entre parents pour un même trait	6
4.3. Transmission des traits inactifs	6
5. Naissance — Pipeline complet	6
6. Génération des pouvoirs & cas spéciaux	7
6.1. Cas spéciaux : mutation forte et naissance sans pouvoir	7
6.2. Suppression des générations	8
6.3. Mutation faible	8
6.4. Algorithme traits → pouvoirs	8
6.4.1. Constitution des sous-listes de traits	9
6.4.2. Transformation d'une sous-liste en pouvoir	11
6.5. Avancement du temps et vieillissement	15
6.6. Tick annuel — reproduction, couples, divorces	15
6.6.1. Consanguinité	16
6.6.2. Portée (nombre d'enfants)	16
6.7. Mort	17
6.8. Création / édition manuelle d'individus	17
7. Puissance et maîtrise	17
7.1. Mutation forte	17
7.2. Naissance normale	17
Exemple 1	18
Exemple 2	18
8. Interface utilisateur	18
8.1. Liste des individus	18
8.2. Fiche d'un individu	19

8.3. Arbre généalogique (page dédiée)	19
8.4. Autres pages	19
8.5. Modes d’affichage des traits d’un individu	19
9. Paramétrage	19
9.1. Paramètres de génération de pouvoir	20
9.2. Paramètres d’hérédité	20
9.3. Paramètres de population	20
9.4. Paramètres d’espèce	20
9.5. Catalogues	21
10. Modes d’utilisation	21
10.1. Avancement du temps (mode principal)	21
10.2. Reproduction manuelle (sandbox uniquement)	21
10.3. Sandbox	21
11. Persistance	22
12. Cible technique	22
13. Points en suspens	22

1. Vue d’ensemble

Le projet est un **générateur de pouvoirs aléatoires** centré sur des **personnes** qui possèdent ces pouvoirs et les transmettent à leur descendance via un **système d’hérédité génétique** porté par un objet **ADN**.

L’application doit permettre :

- de générer une population initiale de personnes (par défaut sans pouvoir) ;
- de faire avancer le temps **année par année**, ce qui déclenche divorces, mises en couple, reproductions et naissances ;
- de visualiser et explorer l’arbre généalogique, les individus, leur ADN, leurs traits, leurs pouvoirs ;
- d’expérimenter librement dans un **mode bac à sable** (sandbox) sans impacter la généalogie réelle ;
- d’exporter et importer toute la configuration et toutes les données générées.

L’ensemble du comportement aléatoire est entièrement **déterministe** : il découle d’une seule **seed** (cf. §2).

2. Déterminisme — Seed unique

Toute l’aléatoire présente dans le programme provient d’une **seule et même seed**, générée aléatoirement et **visible dans les paramètres**.

- La seed est un **entier 64 bits**, **affiché en clair**, **éditable manuellement** par l’utilisateur.

- Un **bouton**, situé sous l’affichage de la seed, permet de **régénérer** une nouvelle seed.
- Un **unique générateur pseudo-aléatoire** est initialisé à partir de cette seed (à la manière d’un `new Random(seed)` en Java), puis transmis de fonction en fonction. **Aucune autre source d’aléatoire** n’est utilisée.
- Objectif : garantir qu’un développeur en débogage puisse **reproduire exactement** le comportement (et donc un bug) à partir de la seed et de la même séquence d’actions.
- La seed fait partie des **données de configuration exportées** (cf. §11) et est **restaurée** lors du rechargement d’une configuration.

Conséquence : la reproductibilité repose sur *même seed + même séquence d’actions utilisateur*. Les données exportées contiennent par ailleurs les résultats déjà produits.

3. Concepts du domaine

3.1. Traits

Un **trait** est l’unité élémentaire qui compose un pouvoir. Il existe **6 types de traits** (types fixés en dur dans le programme ; le contenu de chaque type est éditable) :

Type de trait	Exemples
Remplacements	Pinces de crabe, Mandibule, Tentacules, Halo divin...
Parties du corps	Mains, Bras, Yeux, Peau, Cheveux...
Etats	Lumineux, Visqueux, Gazeux, Froid, Vibrant...
Elements	feu, eau, métaux, ombre, lumière, plume, électricité...
Ajouts	Fourrure, Plume, Tentacules, Carapace, Cristaux...
Actions	contrôle, créé, brule, téléporte, anime, soigne avec...

- Les **listes par défaut** sont celles du dossier `rsrc/ExempleTraits/`.
- Un utilisateur peut **ajouter ou retirer** des traits dans chaque catégorie à l’exécution.
- Un même mot peut apparaître dans plusieurs catégories (ex. *Fleur* dans Remplacements ET dans Ajouts) — chaque occurrence est un trait distinct, indépendant.

3.2. Pouvoirs

Un **pouvoir** est une combinaison de traits, assortie d’une **puissance** et d’une **maîtrise** (cf. §7).

Contrairement à la V1, les « **types de pouvoirs** » **n’existent plus** comme structure permanente. Ils ne subsistent **que pour la génération d’un pouvoir en cas de mutation forte** (cf. §6.1). Dans tous les autres cas, les pouvoirs d’un individu sont **dérivés de son ADN** par l’algorithme de transformation traits → pouvoirs (cf. §6.4).

Les pouvoirs (et leur puissance/maîtrise) sont **calculés une seule fois, à la naissance** de l'individu, puis **stockés** dans sa description (cf. §3.3). Ils ne sont pas recalculés ensuite.

3.3. Personnes

Une **personne** est décrite par les champs suivants :

Champ	Détail
id	Identifiant unique
nom	Généré aléatoirement (éditable)
sexe / genre	Enum paramétrable par espèce (cf. §3.4). Le genre " tout " est présent par défaut chez toutes les espèces.
espèce	Enum appartenant au catalogue des espèces (cf. §3.4). Défaut : " humain ".
date de naissance	Date précise (jour, mois, année). Le temps « courant » est toujours le 1 ^{er} janvier d'une année (cf. §6.5).
âge	Entier = année courante - année de naissance . Tout le monde naît à 0 an .
vivant / décédé	Booléen. Si décédé : champ obligatoire raison du décès (texte libre). Voir §6.7.
parents	Liste de références vers d'autres personnes. null pour le batch initial.
enfants	Liste de références vers les enfants. Mise à jour automatiquement à chaque naissance.
conjoint(s)	Liste de conjoints, chacun marqué actuel ou ex (cf. §6.6).
ADN	Objet ADN : liste de triplets (trait , actif/inactif , résilience) (cf. §3.5). Peut être vide.
pouvoir(s)	Liste de pouvoirs, chacun avec sa puissance et sa maîtrise (cf. §7). Calculée à la naissance. Peut être vide.
notes	Texte libre éditable. Peut être null .

Un individu peut avoir **un nombre variable de pouvoirs**, qui émerge des combinaisons de traits actifs de son ADN.

La notion de **génération** comme champ de la personne **disparaît** (cf. §6.2). Elle ne subsiste que comme **regroupement d'affichage** (tranches de 20 ans), **affiché sur la fiche de l'individu** (cf. §8.2) et utilisé pour les **tris et filtres** (cf. §8.1).

3.4. Espèces et genres

Les **espèces** forment un catalogue éditable par l'utilisateur :

- l'utilisateur peut **ajouter, retirer et modifier** des espèces ;

- la seule espèce présente par défaut est **"humain"** ;
- chaque espèce porte ses propres **paramètres de reproduction et de portée** (cf. §9.4).

Pour chaque espèce, l'utilisateur définit l'**enum des genres / sexes** valides pour cette espèce. Quelle que soit l'espèce, un genre spécial **"tout"** est toujours présent par défaut :

- une personne de genre **"tout"** peut se mettre en couple / se reproduire **avec n'importe quel genre** ;
- en revanche, le genre **"tout"** **ne fait pas d'inter-espèces** et **ne brise pas les couples** existants ;
- seules les personnes **seules** (célibataires ou divorcées) peuvent former un **nouveau** couple (cf. §6.6).

3.5. ADN

L'**ADN** est l'objet central qui décrit la génétique d'une personne. C'est une **liste de triplets** :

(trait, état [actif | inactif], résilience génétique)

- C'est l'ADN qui porte les traits hérités d'un individu (champ **ADN** du §3.3).
- La **résilience** est le pourcentage de chance que le trait soit transmis **actif** à la descendance.
- Un trait peut être présent en ADN tout en étant **inactif** : il est quand même transmissible (cf. §4.3).

4. Hérité — Modèle de résilience

Ce modèle est **inchangé par rapport à la V1** ; seule la terminologie « mutation » devient « **mutation forte** » (cf. §6.1), et la représentation des traits d'un individu est désormais l'objet **ADN** (§3.5).

4.1. Principe

Lorsqu'un enfant naît, **il hérite de la totalité des traits de tous ses parents**. La question n'est donc pas *est-ce qu'il hérite du trait* mais *est-ce que le trait sera actif ou inactif chez lui*.

Chaque trait porté par un individu possède une **résilience** : un pourcentage qui représente sa probabilité d'être transmis **actif** à la descendance.

- Si un trait est tiré **actif** chez l'enfant → un **bonus** est appliqué à sa résilience chez l'enfant.
- Si un trait est tiré **inactif** chez l'enfant → un **malus** est appliqué à sa résilience chez l'enfant.
- Tous les traits ont une **résilience maximale** au-dessus de laquelle le bonus ne s'applique plus.
- Si la résilience tombe sous un **seuil de disparition** (par défaut 2 %), le trait disparaît de l'ADN de l'enfant. **C'est la seule manière pour qu'un trait disparaisse définitivement de la lignée.**

4.2. Combinaison entre parents pour un même trait

Pour chaque parent qui porte le trait X (qu'il l'ait actif ou inactif), on effectue un **tirage individuel selon la résilience que ce parent a pour X** afin de déterminer si **ce parent transmet le trait actif ou inactif à l'enfant**. On agrège ensuite tous les tirages :

Cas 1 — Un seul parent porte le trait :

- Un seul tirage est effectué.
- Si transmis **actif** → trait actif chez l'enfant, résilience initiale = résilience du parent porteur, **bonus** appliqué.
- Si transmis **inactif** → trait inactif chez l'enfant, résilience initiale = résilience du parent porteur, **malus** appliqué.

Cas 2 — Plusieurs parents portent le trait : On regarde le **résultat des tirages** (et non l'état actif / inactif chez les parents) :

- **Aucun tirage ne donne actif** → trait inactif chez l'enfant, résilience initiale = **la plus haute** parmi les parents porteurs, **malus** appliqué.
- **Un seul tirage donne actif** → trait actif chez l'enfant, résilience initiale = résilience du parent dont le tirage est actif, **bonus** appliqué.
- **Plusieurs tirages donnent actif** → trait actif chez l'enfant, résilience initiale = **la plus haute** parmi les parents porteurs, le **bonus est appliqué autant de fois qu'il y a de tirages actifs**.

4.3. Transmission des traits inactifs

Un trait **inactif** chez un individu **est quand même transmis** à ses enfants. La même logique s'applique : tirage actif/inactif selon la résilience, bonus si actif, malus si inactif. Un génome peut donc « se réveiller » plusieurs générations plus tard.

5. Naissance — Pipeline complet

À chaque naissance, le traitement suit cet ordre :

1. **Tirage du cas spécial** (cf. §6.1), avec leurs taux respectifs :
 - **mutation forte**, ou
 - **naissance sans pouvoir**, ou
 - **naissance normale** (cas par défaut).
2. **Constitution de l'ADN** :
 - *Naissance normale* : hérédité standard (§4) à partir de tous les parents.
 - *Mutation forte / sans pouvoir* : tous les traits parentaux sont hérités **inactifs** (cf. §6.1).
3. **Mutation faible** (cf. §6.3) — **uniquement pour les naissances normales** : tirages indépendants de gain et de perte de trait, appliqués à l'ADN.

4. Calcul des pouvoirs :

- *Naissance normale* : algorithme traits → pouvoirs (§6.4) sur les traits **actifs** de l'ADN.
- *Mutation forte* : un unique pouvoir issu du gabarit tiré (§6.1).
- *Sans pouvoir* : aucun pouvoir.

5. Calcul de la puissance et de la maîtrise de chaque pouvoir (§7).

6. Génération des pouvoirs & cas spéciaux

6.1. Cas spéciaux : mutation forte et naissance sans pouvoir

À chaque naissance, deux événements probabilistes peuvent se produire (chacun avec un taux paramétrable) :

Mutation forte (nouveau nom de la « mutation » V1) :

- Tous les traits parentaux sont hérités **inactifs**.
- Un **nouveau pouvoir aléatoire** est tiré selon le gabarit ci-dessous.
- Ses traits constitutifs sont **actifs** avec une **résilience initiale paramétrable**.

Gabarit de génération du pouvoir de mutation forte (c'est le **seul** endroit où les « types de pouvoirs » subsistent) :

Un nombre aléatoire **i** entre **0 et 5** est tiré. Selon **i**, le pouvoir associe :

- **AE** (cas les plus fréquents, **i** ∈ {0, 1, 2}) : une **Action** + un **Élément**.
- **PE** (**i** = 3) : une **Partie du corps** + un **État**.
- **PA** (**i** = 4) : une **Partie du corps** + un **Ajout**.
- **PR** (**i** = 5) : une **Partie du corps** + un **Remplacement**.

Chaque trait est tiré aléatoirement dans la liste des traits de son type, **avec les pondérations** (§9.1).

Le **libellé lisible** du pouvoir dépend du gabarit tiré :

- **AE** → « {action} {élément} »
- **PE** → « {partie du corps} {état} »
- **PA** → « {ajout} sur {partie du corps} »
- **PR** → « {remplacement} à la place de {partie du corps} »

Le type **AEP** (Action + Élément + Partie du corps) de la V1 **disparaît** du gabarit. Il pourra toutefois **émerger** de façon dérivée via l'algorithme traits → pouvoirs (§6.4).

Naissance sans pouvoir :

- Tous les traits parentaux sont hérités **inactifs**.
- Aucun pouvoir n'est tiré pour l'enfant.
- Ses descendants peuvent quand même réactiver les traits.

Option globale : malus sur le génome en cas de mutation forte / enfant sans pouvoir

- Activable / désactivable. **Par défaut désactivée** : les traits inactifs sont hérités sans pénalité de résilience.
- Dans ces deux cas spéciaux, si plusieurs parents partagent un trait, on conserve simplement la **résilience la plus élevée**.

6.2. Suppression des générations

La notion de **génération** (champ de personne, numérotation, $\max(\text{parents})+1$) **disparaît**. Le temps est désormais géré par des **dates** et l'**avancement par années** (§6.5).

Subsiste uniquement une **génération d'affichage** = tranche de **20 ans** de l'année de naissance (par ex. génération 0 = années [0, 19], génération 1 = [20, 39], etc.). Elle est **calculée à partir de l'année de naissance, affichée sur la fiche de l'individu** (cf. §8.2) et sert aux **tris et filtres** (cf. §8.1).

6.3. Mutation faible

Les mutations faibles **ne concernent que les naissances normales** (ni *sans pouvoir*, ni *mutation forte*). À la naissance, deux tirages **indépendants** (pouvant se produire **simultanément**) :

Gain d'un trait — avec un pourcentage paramétrable, la personne gagne **un trait tiré au hasard** dans la liste des traits :

- si elle **possède déjà** ce trait → on le met **actif** et on applique le **bonus** de résilience ;
- si elle **ne le possède pas** → on l'ajoute **actif, sans** bonus de résilience, avec la **résilience initiale paramétrée**.

Perte d'un trait — avec un **autre** pourcentage paramétrable, la personne **perd un trait** tiré au hasard parmi **ses** traits (actif ou inactif).

6.4. Algorithme traits → pouvoirs

Cet algorithme construit les pouvoirs à partir de la liste des **traits actifs** d'une personne. Il fait intervenir **deux paramètres distincts**, tous deux **réglables en paramètre** :

- la **constante de duplication** **D** (cf. §6.4.1), qui pilote la probabilité de duplication des traits secondaires ;
- la **constante de génération** **K** (cf. §6.4.2), qui pilote la probabilité de génération de nouveaux traits (**Ka**, **Ke**, ...).

Si la personne n'a **aucun trait actif**, elle est **sans pouvoir**.

6.4.1. Constitution des sous-listes de traits

On part de la liste de tous les traits **actifs**.

- S'il existe des traits de type **Action** : ce sont les **traits principaux**, les autres sont **secondaires**.
- Sinon, s'il existe des traits de type **Partie du corps** (et aucune Action) : les **Parties du corps** sont les **traits principaux**, les autres sont **secondaires**.
- Sinon (ni Action ni Partie du corps) : il n'y a **qu'une seule sous-liste** contenant tous les traits.

On crée **autant de sous-listes que de traits principaux**, chacune initialisée avec un trait principal. Puis on répartit les traits secondaires :

1. On place les secondaires dans une liste et les principaux dans une liste.
2. On **mélange aléatoirement** (selon la seed) la liste des secondaires **et** celle des principaux.
3. On parcourt : 1□□ secondaire avec 1□□ principal, 2□ avec 2□, etc.
 - S'il y a **plus de principaux que de secondaires**, certains principaux restent sans secondaire.
 - S'il y a **plus de secondaires que de principaux**, on **recommence le parcours des principaux** depuis le début, dans le même ordre.

Duplication : au moment où l'on assigne un trait secondaire, il y a (**résilience du trait secondaire / D**) % de chance que ce trait **se duplique**, c'est-à-dire qu'une **copie** soit aussi placée dans **une autre** sous-liste. (**D** = constante de duplication, distincte de la constante de génération **K** du §6.4.2.)

- Un trait dupliqué ne peut **pas apparaître deux fois dans une même sous-liste** ; il peut donc être dupliqué au maximum autant de fois qu'il y a de sous-listes.
- La duplication **ne modifie pas l'ADN** : elle n'existe que pour la construction des pouvoirs.

Exemple 1 — sans duplication

Une personne a en traits actifs 2 actions (**a1**, **a2**), 1 état (**e1**) et 3 remplacements (**r1**, **r2**, **r3**).

- Principaux : **a1**, **a2** ; secondaires : **e1**, **r1**, **r2**, **r3**.
- Assignment un à un :

[a1]	[e1, r1, r2, r3]
[a2]	
[a1, e1]	[r1, r2, r3]

```

[ a2 ]

[ a1, e1 ]      [ r2, r3 ]
[ a2, r1 ]

[ a1, e1, r2 ]   [ r3 ]
[ a2, r1 ]

[ a1, e1, r2 ]
[ a2, r1, r3 ]

```

Exemple 2 — avec duplication

Une personne a 3 actions (a1, a2, a3), 4 états (e1...e4) et 3 remplacements (r1, r2, r3).

```

[ a1 ]                [ e1, e2, e3, e4, r1, r2, r3 ]
[ a2 ]
[ a3 ]

[ a1, e1 ]            [ e2, e3, e4, r1, r2, r3 ]
[ a2 ]
[ a3 ]

[ a1, e1 ]            [ e2, e3, e4, r1, r2, r3 ] (e2 se duplique)
[ a2, e2 ]
[ a3 ]

[ a1, e1 ]            [ e3, e4, r1, r2, r3 ]
[ a2, e2 ]
[ a3, e2 ]

[ a1, e1, e3 ]        [ e4, r1, r2, r3 ]
[ a2, e2 ]
[ a3, e2 ]

[ a1, e1, e3 ]        [ r1, r2, r3 ]
[ a2, e2, e4 ]
[ a3, e2 ]

[ a1, e1, e3 ]        [ r1, r2, r3 ] (r1 se duplique)
[ a2, e2, e4 ]
[ a3, e2, r1 ]

[ a1, e1, e3, r1 ]    [ r1, r2, r3 ] (r1 a déjà 3 occurrences = nb de sous-listes →
ne peut plus se dupliquer)
[ a2, e2, e4 ]
[ a3, e2, r1 ]

[ a1, e1, e3, r1 ]    [ r2, r3 ]
[ a2, e2, e4, r1 ]

```

```
[ a3, e2, r1 ]

[ a1, e1, e3, r1 ]    [ r3 ]
[ a2, e2, e4, r1 ]
[ a3, e2, r1, r2 ]

[ a1, e1, e3, r1, r3 ] [ ]
[ a2, e2, e4, r1 ]
[ a3, e2, r1, r2 ]
```

6.4.2. Transformation d'une sous-liste en pouvoir

Si une sous-liste contient **plusieurs traits du même type**, on les considère comme **un seul** trait en les séparant par des virgules + « et » (ou « ou » pour les états). Cela ne change **rien à l'ADN**, c'est purement pour l'affichage du pouvoir.

- Plusieurs Remplacements ($r1, r2, r3$) → « $r1, r2$ et $r3$ ».
- Plusieurs États ($e1, e2, e3, e4$) → « $e1$ ou $e2$ ou $e3$ ou $e4$ ».

Notations :

- **a** = trait/groupe de type **Action** ; **e** = **Élément** ; **p** = **Partie du corps** ; **aj** = **Ajout** ; **r** = **Remplacement** ; **et** = **État**.
- **if a** : signifie « s'il y a au moins une action dans la sous-liste ».
- **Ka / Ke / Kp / Kaj / Kr / Ket** : **K % de chance** de générer **UN SEUL** nouveau trait du type indiqué (**K** = constante de génération, distincte de la constante de duplication **D** du §6.4.1).
 - On tente une génération via **K** autant de fois que la notation **K...** apparaît dans le pouvoir choisi.
 - Les traits ainsi générés sont **inscrits dans l'ADN** (en actif). S'il existe déjà dans l'ADN, on le met **actif + bonus** de résilience.
 - **Si le tirage K échoue** : c'est comme **pouvoir = null** → **aucun pouvoir** n'est produit par cette sous-liste (mais le trait reste présent et actif dans l'ADN, sauf s'il s'agissait d'une duplication).

On applique **une fois par sous-liste** l'algorithme suivant.



Arbre faisant foi, reproduit verbatim depuis **PromptDescriptionV2.md** et **validé par l'auteur**. Les comportements pouvant sembler « incohérents » (état non repris dans certaines branches $a \ \& \ e \ \& \ \neg p$, $\{Ka\}$ absent de certaines feuilles $\neg a \ \& \ \neg p \ \& \ \neg e$, etc.) sont **volontaires**.

```
if a :
  if e :
    if p :
      if r :
        if aj :
          if et :
```

```

        pouvoir = "{a} {e} avec {aj}, {et} sur {r} à la place de {p}"
    else :
        pouvoir = "{a} {e} avec {aj} sur {r} à la place de {p}"
else :
    if et :
        pouvoir = "{a} {e} avec {r} {et} à la place de {p}"
    else :
        pouvoir = "{a} {e} avec {r} à la place de {p}"
else :
    if aj :
        if et :
            pouvoir = "{a} {e} avec {aj} {et} sur {p}"
        else :
            pouvoir = "{a} {e} avec {aj} sur {p}"
    else :
        if et :
            pouvoir = "{a} {e} avec {p} {et}"
        else :
            pouvoir = "{a} {e} avec {p}"
else:
    if r:
        if aj:
            if et:
                pouvoir = "{a} {e} avec {aj} sur {r}"
            else:
                pouvoir = "{a} {e} avec {aj} sur {r}"
        else:
            if et:
                pouvoir = "{a} {e} avec {r}"
            else:
                pouvoir = "{a} {e} avec {r}"
    else:
        if aj:
            if et:
                pouvoir = "{a} {e} avec {aj}"
            else:
                pouvoir = "{a} {e} avec {aj}"
        else:
            if et:
                pouvoir = "{a} {e} {et}"
            else:
                pouvoir = "{a} {e}"
else:
    if aj:
        if et:
            if r:
                if p:
                    pouvoir = "{a} {aj} {et} avec {r} à la place de {p}"
                else:
                    pouvoir = "{a} {aj} {et} sur {r}"
            else:

```

```

        if p:
            pouvoir = "{a} {aj} {et} avec {p}"
        else:
            pouvoir = "{a} {aj} {et}"
    else:
        if r:
            if p:
                pouvoir = "{a} {aj} avec {r} à la place de {p}"
            else:
                pouvoir = "{a} {aj} sur {r}"
        else:
            if p:
                pouvoir = "{a} {aj} avec {p}"
            else:
                pouvoir = "{a} {aj}"
    else:
        if r:
            if et:
                if p:
                    pouvoir = "{a} {r} {et} avec {p}"
                else:
                    pouvoir = "{a} {r} {et}"
            else:
                if p:
                    pouvoir = "{a} {r} avec {p}"
                else:
                    pouvoir = "{a} {r}"
        else:
            if p:
                if et:
                    pouvoir = "{a} {p} {et}"
                else:
                    pouvoir = "{a} {p}"
            else:
                if et:
                    pouvoir = "{a} {Ke} {et}"
                else:
                    pouvoir = "{a} {Ke}"
    else:
        if p:
            if aj:
                if r:
                    if et:
                        if e:
                            pouvoir = "{aj} {et} sur {r} en {e} à la place de {p}"
                        else:
                            pouvoir = "{aj} {et} sur {r} à la place de {p}"
                    else:
                        if e:
                            pouvoir = "{aj} en {e} sur {r} à la place de {p}"
                        else:

```

```

        pouvoir = "{aj} sur {r} à la place de {p}"
    else:
        if et:
            if e:
                pouvoir = "{aj} {et} en {e} sur {p}"
            else:
                pouvoir = "{aj} {et} sur {p}"
        else:
            if e:
                pouvoir = "{aj} en {e} sur {p}"
            else:
                pouvoir = "{aj} sur {p}"
    else:
        if r:
            if et:
                if e:
                    pouvoir = "{r} {et} en {e} à la place de {p}"
                else:
                    pouvoir = "{r} {et} à la place de {p}"
            else:
                if e:
                    pouvoir = "{r} en {e} à la place de {p}"
                else:
                    pouvoir = "{r} à la place de {p}"
        else:
            if et:
                if e:
                    pouvoir = "{p} {et} en {e}"
                else:
                    pouvoir = "{p} {et}"
            else:
                if e:
                    pouvoir = "{p} en {e}"
                else:
                    pouvoir = "{p} {Kaj}"
    else:
        if e:
            if et:
                if aj:
                    if r:
                        pouvoir = "{Ka} {e} avec {aj} {et} sur {r} à la place de {Kp}"
                    else:
                        pouvoir = "{Ka} {e} avec {aj} {et} sur {Kp}"
                else:
                    if r:
                        pouvoir = "{Ka} {e} avec {r} {et} à la place de {Kp}"
                    else:
                        pouvoir = "{Ka} {e} {et}"
            else:
                if aj:
                    if r:

```

```

        pouvoir = "{Ka} {e} avec {aj} sur {r} à la place de {Kp}"
    else:
        pouvoir = "{Ka} {e} avec {aj} sur {Kp}"
    else:
        if r:
            pouvoir = "{Ka} {e} avec {r} à la place de {Kp}"
        else:
            pouvoir = "{Ka} {e}"
    else:
        if aj:
            if et:
                if r:
                    pouvoir = "{Ka} {aj} {et} avec {r} à la place de {Kp}"
                else:
                    pouvoir = "{aj} {et} sur {Kp}"
            else:
                if r:
                    pouvoir = "{aj} sur {r} à la place de {Kp}"
                else:
                    pouvoir = "{aj} à la place de {Kp}"
        else:
            if r:
                if et:
                    pouvoir = "{r} {et} à la place de {Kp}"
                else:
                    pouvoir = "{r} à la place de {Kp}"
            else:
                if et:
                    pouvoir = "{Kp} {et}"
                else:
                    pouvoir = null

```

6.5. Avancement du temps et vieillissement

- Le temps « courant » est **toujours le 1^{er} janvier** d'une année donnée.
- Le bouton « **reproduire tout** » de la V1 **disparaît**, remplacé par un bouton « **avancer de X années** » (X **réglable**, minimum **1 an**). Avancer fait **défiler la date** (année par année) et applique, **pour chaque année**, le **tick annuel** (§6.6).
- **Plus personne ne meurt naturellement** : il n'y a plus d'âge de mort, et l'**option « immortel » disparaît**. Seul l'utilisateur peut tuer un individu (cf. §6.7).
- Tout le monde **naît à 0 an**. L'âge = **année courante** - **année de naissance**.
- Les nouveau-nés d'une année reçoivent une **date de naissance** = un **jour précis tiré aléatoirement** dans l'année de naissance.

6.6. Tick annuel — reproduction, couples, divorces

Chaque année avancée applique, dans l'ordre :

1. Divorces potentiels. Pour chaque couple, selon le **% de divorce de l'espèce** (testé à **chaque année**), le couple peut se dissoudre : ses membres **redeviennent célibataires** et leurs anciens conjoints deviennent des « **ex** ».

2. Mises en couple et naissances.

- Chaque personne **célibataire ou divorcée**, et **pas plus vieille que l'âge maximal de reproduction** de son espèce, tire individuellement, selon la **gaussienne de reproduction** (§9.4), le **% de chance qu'elle veuille se reproduire** cette année.
- On constitue la **liste des candidats** et on tire **aléatoirement des groupes** de la **taille requise par l'espèce**, en respectant les **règles de consanguinité** (§6.6.1) et **sans inter-espèces**.
 - Les candidats n'ayant **pas trouvé assez de partenaires** sont **notés** et **re-candidatent l'année suivante**.
 - Un groupe ainsi formé devient un **couple** : ses membres deviennent **conjoints actuels** les uns des autres et ne peuvent se reproduire **qu'au sein de ce groupe** tant qu'il n'y a pas divorce.
- Les **couples déjà formés** ont chaque année un **% de chance de se reproduire issu de la même gaussienne de reproduction** (§9.4) que celle des candidats célibataires (évaluée selon l'âge des membres du couple) ; ce pourcentage reste **éditable couple par couple** par l'utilisateur.
- Toute reproduction produit une **portée** (§6.6.2).

Le genre "**tout**" peut être groupé avec n'importe quel genre, mais **jamais en inter-espèces** et **sans briser** de couple existant (§3.4).

6.6.1. Consanguinité

Un **paramètre** autorise ou non la **consanguinité** (par défaut : **non autorisée**).

Si la consanguinité est **interdite** : deux personnes qui partagent **les mêmes parents OU les mêmes grands-parents** ne peuvent **pas former de couple** (et donc ne peuvent pas se reproduire ensemble).

6.6.2. Portée (nombre d'enfants)

Le nombre d'enfants d'une reproduction dépend de l'espèce, via trois paramètres (cf. §9.4) :

- **M** : nombre **minimum** d'enfants (plancher) ;
- **N** : nombre **maximum** d'enfants (plafond) ;
- **X %** : chance de tirer **un enfant supplémentaire**.

Procédure : on part de **M** enfants garantis ; puis, **à partir du plancher**, on tire à **X %** « y a-t-il un autre enfant ? ». On **refait le tirage** tant que la réponse est « oui », **jusqu'à un échec ou jusqu'à atteindre N**.

6.7. Mort

- Plus de **mort naturelle**, plus d'âge de mort, plus d'option « immortel ».
- L'utilisateur peut **tuer manuellement** un individu, **quel que soit son âge**, en renseignant **obligatoirement** une **cause de mort** (texte libre stocké dans **raison du décès**).
- Un individu mort reste dans l'arbre généalogique, marqué « décédé », et ne peut plus se reproduire.

6.8. Création / édition manuelle d'individus

L'utilisateur peut :

- créer un **nouvel individu personnalisé** en renseignant librement tous les champs (espèce, genre, ADN, pouvoirs, notes...);
- créer un **nouvel individu basé sur un individu existant** (clonage éditable).

7. Puissance et maîtrise

Chaque pouvoir possède une **puissance** et une **maîtrise** (valeurs entières). Elles ne sont **bornées à [1, 10] que lorsqu'une nouvelle valeur aléatoire est tirée** (mutation forte, ou cas *A* d'une naissance — cf. §7.2). Dans tous les autres cas (valeurs dérivées de la moyenne des parents), **il n'y a aucune borne**, ni inférieure ni supérieure : par exemple, des parents de puissance moyenne 10 peuvent donner un enfant de puissance 11.

7.1. Mutation forte

En cas de mutation forte, **puissance** et **maîtrise** sont deux nombres **aléatoires entre 1 et 10** (bornes incluses).

7.2. Naissance normale

Les pouvoirs de l'enfant ont été calculés (§6.4). Pour leur attribuer puissance/maîtrise :

- Avant le calcul, on **mélange aléatoirement** (selon la seed) la liste des pouvoirs **de chaque parent**.
- Pour le **i -^{ème} pouvoir** de l'enfant, on calcule la **moyenne** (de la puissance, puis de la maîtrise) des **i -^{èmes} pouvoirs de tous les parents qui ont au moins un pouvoir**.
 - Si un parent n'a pas de i -^{ème} pouvoir, on prend son **i modulo (nombre de pouvoirs du parent)-^{ème} pouvoir**.
- La **moyenne** est arrondie ainsi : soit x la moyenne et n un entier, $x \geq n + 0,5 \Rightarrow x = n + 1$; $x < n + 0,5 \Rightarrow x = n$.
- On tire ensuite la valeur finale (indépendamment pour puissance et pour maîtrise, avec les **mêmes probabilités**) :

- **A %** → une **nouvelle valeur aléatoire 1..10** ;
- **B %** → **moyenne + 1** ;
- **C %** → **moyenne** ;
- **B %** → **moyenne + 1**.
- **B et C** sont **réglables en paramètre** ; **A = 100 - 2·B - C** est **affichée** à côté.
- **Bornage** : seul le cas **A** (nouvelle valeur aléatoire) est borné à **[1, 10]**. Les cas **moyenne + 1**, **moyenne** et **moyenne + 1** **ne sont pas bornés** — la valeur peut donc dépasser 10 (ou être inférieure à 1).

Exemple 1

Enfant à 3 pouvoirs (**pe1**, **pe2**, **pe3**). Parents : l'un avec 2 pouvoirs (**pa1**, **pa2**), l'autre avec 3 (**pb1**, **pb2**, **pb3**).

- **pe1** ← moyenne de **pa1** et **pb1**.
- **pe2** ← moyenne de **pa2** et **pb2**.
- **pe3** ← moyenne de **pa1** (car $3 \bmod 2 = 1 \rightarrow 1$ pouvoir) et **pb3**.

Exemple 2

Enfant à 2 pouvoirs (**pe1**, **pe2**). Parents : l'un avec 4 pouvoirs, l'autre avec 3.

- **pe1** ← moyenne de **pa1** et **pb1**.
- **pe2** ← moyenne de **pa2** et **pb2**.
- **pa3**, **pa4**, **pb3** ne sont pas utilisés.

8. Interface utilisateur

L'application propose plusieurs **pages** distinctes.

8.1. Liste des individus

Page qui liste **tous les individus** avec recherche et filtres (par **génération** [tranches de 20 ans], espèce, trait, pouvoir, statut vivant/décédé, etc.). Chaque ligne / carte affiche :

- **nom**,
- **date de naissance** et **âge**,
- **pouvoir(s)**.

Un clic sur un individu ouvre sa **fiche individu** (§8.2).

8.2. Fiche d'un individu

Page dédiée à un individu, qui contient :

- ses **informations globales** (nom, date de naissance, âge, **génération** [tranche de 20 ans, cf. §6.2], espèce, genre, statut vivant/décédé, notes, parents, conjoints...) ;
- son **ADN** / **ses traits**, affichés selon le mode d'affichage actif (cf. §8.5) ;
- ses **pouvoirs**, avec **puissance** et **maîtrise** ;
- un **arbre généalogique** centré sur lui, avec une **profondeur N sélectionnable** (par défaut 2) : N niveaux d'ancêtres au-dessus et N niveaux de descendants au-dessous, dans la mesure du possible.

Chaque case de l'arbre contient **nom, date de naissance / âge et pouvoir(s)**. Un clic sur une case ouvre la fiche de l'individu cliqué (recentrage).

8.3. Arbre généalogique (page dédiée)

Page qui affiche **le même arbre** que la fiche (§8.2), centré sur le même individu, même profondeur N, mais **sans informations latérales** : seulement l'arbre, en grand. Un clic sur une case recentre l'arbre.

8.4. Autres pages

- **Écran d'avancement du temps** : bouton « **avancer de X années** » (X réglable, min. 1).
- **Écran sandbox** : voir §10.3. La **reproduction manuelle** (sélection de 1, 2 ou plusieurs individus) n'est disponible **qu'en sandbox**.
- **Page des paramètres** : voir §9 ; affiche notamment la **seed** (+ bouton de régénération) et les **courbes gaussiennes** de reproduction par espèce.

8.5. Modes d'affichage des traits d'un individu

Trois modes au choix :

- **Mode 1** : pouvoirs uniquement ;
- **Mode 2** : pouvoirs + traits actifs ;
- **Mode 3** : pouvoirs + traits actifs + traits inactifs + résilience de chaque trait.

9. Paramétrage

Tout le système doit être **paramétrable**. La plupart des paramètres se déclinent à trois niveaux : **global, par type de trait, par trait individuel**.

9.1. Paramètres de génération de pouvoir

- **Seed** (cf. §2).
- Poids de chaque **type de trait** dans les tirages.
- Poids de chaque **trait individuel** dans les tirages.
- Résilience initiale attribuée aux traits d'un pouvoir nouvellement tiré (mutation forte).
- Constante de **duplication D** : pilote la duplication des traits secondaires (§6.4.1).
- Constante de **génération K** : pilote la génération de nouveaux traits $K \cdots$ (§6.4.2). **Distincte de D**.

9.2. Paramètres d'hérédité

- Résilience initiale : globale, par type de trait, par trait.
- Résilience maximale : globale, par type de trait, par trait.
- Facteur de **bonus** (trait tiré actif).
- Facteur de **malus** (trait tiré inactif).
- **Seuil de disparition** (par défaut 2 %).
- Taux de **mutation forte** par naissance.
- Taux de **naissance sans pouvoir** par naissance.
- Taux de **mutation faible** — **gain** d'un trait par naissance.
- Taux de **mutation faible** — **perte** d'un trait par naissance.
- Option **activer/désactiver le malus** sur le génome en cas de mutation forte / enfant sans pouvoir (désactivé par défaut).
- Probabilités **B** et **C** de puissance/maîtrise (§7) ; $A = 100 - 2 \cdot B - C$ affichée.

9.3. Paramètres de population

- Nombre d'individus du **batch initial**.
- **Année de naissance** du batch initial (par défaut **an 0** ; modifiable **avant** la naissance du batch).
- **% de chance qu'un individu du batch initial possède un pouvoir** (par défaut **0 %**).
- **X** : pas d'avancement du temps (« avancer de X années », min. 1).

9.4. Paramètres d'espèce

Pour chaque espèce :

- Enum des **genres / sexes** (avec "**tout**" toujours présent).
- **Portée** : **M** (min), **N** (max), **X %** (chance d'enfant supplémentaire).
- **Gaussienne de reproduction** (affichée sur la page des paramètres), définie par :
 - **âge de début** de reproduction,

- **âge de fin** de reproduction (âge maximal),
- **âge du pic** (chance de reproduction maximale),
- **probabilité au pic**,
- **pente** : slider d'écart-type avec effet direct sur la courbe affichée.
- % de divorce par an.
- % de reproduction d'un couple déjà formé par an : issu de la même gaussienne de reproduction ci-dessus (pas de paramètre séparé), mais **éditable couple par couple**.

Plus de durée de vie, plus d'âge de mort, plus d'âge à la naissance (toujours 0), plus d'immortalité : ces paramètres V1 **disparaissent**.

9.5. Catalogues

- Listes de traits éditables (ajout / suppression dans chaque type). Listes par défaut : [rsrc/ExempleTraits/](#).
- Catalogue des **espèces** (et leurs genres).
- **Option consanguinité** (autorisée / interdite ; par défaut interdite).
- Les **6 types de traits** sont **fixés** dans la V2. Les « types de pouvoirs » ne subsistent que pour la mutation forte (§6.1).

10. Modes d'utilisation

10.1. Avancement du temps (mode principal)

L'utilisateur fait **avancer le temps de X années**. Chaque année déclenche le **tick annuel** (§6.6) : divorces, mises en couple, reproductions et naissances, et le vieillissement de toute la population.

10.2. Reproduction manuelle (sandbox uniquement)

La sélection manuelle de **1, 2 ou plusieurs** individus pour déclencher une reproduction n'est disponible **qu'en sandbox** (§10.3).

10.3. Sandbox

Un mode bac à sable qui permet de :

- créer des individus temporaires ;
- les faire reproduire entre eux (manuellement) ou avec des individus réels ;
- explorer des scénarios sans affecter la généalogie réelle ;
- valider via un bouton « **make it real** » qui injecte les modifications du bac à sable dans la généalogie principale.

11. Persistance

- **Aucune sauvegarde automatique** côté navigateur.
- L'utilisateur peut **exporter** trois types de fichiers (JSON conseillé) :
 1. **Configuration seule** : tous les paramètres, **la seed**, catalogues de traits, catalogue d'espèces et leurs genres, pondérations, facteurs bonus/malus, seuils, etc.
 2. **Données générées seules** : tous les individus, leur ADN, leurs pouvoirs (puissance/maîtrise), la généalogie, les couples, l'historique.
 3. **Fichier combiné** : configuration + données.
- Chaque fichier contient un **identifiant de type** (ex. `kind: "config" | "data" | "full"` en tête du JSON) qui permet, **à l'import**, de reconnaître automatiquement le type de fichier et d'appliquer le bon traitement.
- Le même format permet de partager un état (entier ou partiel) entre appareils ou entre utilisateurs.

12. Cible technique

Pour permettre l'usage sous **Windows, Mac, Linux**, et sur **smartphones (iOS et Android)**, le projet sera développé comme une **application web (PWA — Progressive Web App)** :

- accessible via navigateur, sans installation obligatoire ;
- installable sur le téléphone comme une app native (icône, plein écran) ;
- aucun backend obligatoire pour la V1 (tout tourne côté client) ;
- persistance exclusivement par **export / import de fichier**.

13. Points en suspens

À préciser plus tard :

1. **Sauvegarde de l'état du générateur aléatoire** : faut-il persister la position du RNG (nombre de tirages consommés) dans l'export de données pour une reprise au tirage près, ou la seed seule suffit-elle ?
2. **Détails de l'appariement** : gestion fine des candidats non appariés sur plusieurs années, ordre de priorité.
3. **Statistiques** affichées lors de l'avancement du temps.
4. **Format précis** du fichier d'export / import.
5. **Maquettes UI** des différents écrans (dont l'affichage des courbes gaussiennes).